

Les outils et les stratégies des tests de sécurité

Référentiel des **tests de sécurité** mis en œuvre pour préparer le déploiement sécurisé de **Hublot** (DIRM Méditerranée).

Ce document complète :

- [SECURITE_4_DEPLOIEMENT.md](../1.2 Préparer le déploiement d'une application/1.2.4.2 Sécurité du déploiement.md) — mesures de sécurité du déploiement
- [SECURITE_AUDIT.md](../1.2 Préparer le déploiement d'une application/1.2.4.3 Audit des dépendances.md) — politique d'audit des dépendances
- [SCENARIOS_TEST.md](../1.2 Préparer le déploiement d'une application/1.2.2 Elaborer un scénario de test.md) — scénarios (dont ST-SEC01, ST-SEC02)

1. Pourquoi tester la sécurité ?

Tester la sécurité, c'est **chercher activement les failles avant un attaquant**. On ne se contente pas de vérifier que l'application fonctionne (tests fonctionnels) : on vérifie qu'elle **résiste** aux usages malveillants et qu'aucune donnée RH sensible ne fuite.

Pour Hublot, les **enjeux** sont :

Enjeu	Risque concret
Données RH nominatives	Fuite d'informations personnelles (RGPD)
Accès au tableau de bord	Consultation par une personne non habilitée
Chaîne d'approvisionnement (npm)	Dépendance vulnérable embarquée en production
Secrets (mots de passe, tokens)	Commit accidentel dans le dépôt Git
Configuration du déploiement	

Enjeu	Risque concret
	En-têtes HTTP absents → clickjacking, MIME sniffing

2. La stratégie : « shift-left » + défense en profondeur

La stratégie retenue applique deux principes complémentaires :

1. **Shift-left** : tester **au plus tôt** et **en continu**, dès le commit et à chaque intégration (CI), plutôt qu'une seule fois avant la mise en production.
2. **Défense en profondeur** : croiser **plusieurs familles d'outils** (chacune couvre un angle), car aucun outil seul ne détecte tout.

Les familles de tests de sécurité

Famille	Question posée	Quand
SCA (Software Composition Analysis)	Mes dépendances ont-elles des failles connues (CVE) ?	À chaque CI + hebdomadaire
SAST (Static Application Security Testing)	Mon code source contient-il des motifs dangereux ?	À chaque push + hebdomadaire
Secret scanning	Ai-je commité un secret (mot de passe, clé) ?	À chaque push
Tests unitaires de sécurité	Mes garde-fous applicatifs (sanitization, session) fonctionnent-ils ?	À chaque CI
Tests de configuration	Les en-têtes HTTP de sécurité sont-ils bien servis ?	Après déploiement
DAST (Dynamic Application Security Testing)	L'application en cours d'exécution est-elle attaquable ?	Évolution (voir §6)

3. Les outils retenus et leur rôle

Outil	Famille	Rôle dans Hublot	Intégration
<code>npm audit / audit:prod</code>	SCA	Détecte les CVE des dépendances de production (critical bloquant, high filtré par allowlist)	CI bloquante (<code>ci.yml</code>) + hebdo (<code>audit-scheduled.yml</code>)
Dependabot	SCA préventif	Ouvre des PR de mise à jour des dépendances npm + actions GitHub	<code>.github/dependabot.yml</code> (hebdo)
Trivy	SCA	Scan complémentaire des vulnérabilités (système de fichiers)	<code>security-scan.yml</code> (informatif)
CodeQL	SAST	Analyse statique JS/TS : injections, XSS, flux de données dangereux	<code>codeql.yml</code> → onglet Security
Gitleaks	Secret scanning	Détecte les secrets dans l'arborescence (allowlist pour les identifiants de démo)	<code>security-scan.yml</code> (bloquant)
Vitest (<code>security.test.ts</code>)	Tests unitaires sécurité	Vérifie <code>sanitizeInput</code> , <code>validateFilter</code> , masquage RH, session/expiration	CI <code>unit-tests</code> (<code>test:run</code>)

Outil	Famille	Rôle dans Hublot	Intégration
Script <code>curl</code> (<code>check-security-headers.sh</code>)	Test de configuration	Vérifie HTTPS + en-têtes de sécurité d'un déploiement (ST-SEC01)	CLI post-déploiement
Playwright (login, session)	Test fonctionnel de sécurité	Vérifie le contrôle d'accès (connexion refusée / session)	CI <code>e2e</code>
ESLint + TypeScript	Qualité préventive	Réduit les motifs à risque (typage strict, lint)	CI <code>lint</code>

4. Comment exécuter ces tests

En local (poste développeur)

```
# Tests unitaires de sécurité (sanitization, session, masquage)
npm run security:test

# Audit des dépendances de production (comme en CI)
npm run audit:prod

# Vérifier les en-têtes de sécurité d'un déploiement (ST-SEC01)
npm run security:headers                # cible la PROD par défaut
npm run security:headers https://mon-url # cible une URL précise
```

En intégration continue (GitHub Actions)

Workflow	Déclenchement	Contenu sécurité
<code>ci.yml</code>	push / PR <code>main</code> , <code>staging</code>	lint, tests unitaires (dont sécurité) , audit npm bloquant, build, E2E (login/session)
<code>codeql.yml</code>	push / PR + hebdo	SAST CodeQL

Workflow	Déclenchement	Contenu sécurité
<code>security-scan.yml</code>	push / PR + hebdo	Gitleaks (bloquant) + Trivy (informatif)
<code>audit-scheduled.yml</code>	hebdomadaire	audit npm + artefact

Principe de blocage : ce qui est **certain et maîtrisé** bloque la livraison (audit npm, secrets). Ce qui est **informatif / à trier** (Trivy, alertes CodeQL) remonte dans l'onglet Security sans casser la CI, pour être traité en revue.

5. Commandes pour la démo (jury)

Ordre conseillé pour une démonstration orale. Chaque commande correspond à un scénario (colonne de droite).

#	Commande	Ce que ça démontre	Scénario
1	<code>npm run security:test</code>	Les garde-fous applicatifs (sanitization anti-XSS, validation des filtres, masquage RH, expiration de session) sont testés automatiquement	ST-SEC02
2	<code>npm run audit:prod</code>	Aucune dépendance de production critical/high non maîtrisée	ST-SEC04
3	<code>npm run security:headers</code>	Les en-têtes HTTP de sécurité de la PROD sont présents (HTTPS, X-Frame-Options, nosniff...)	ST-SEC01
4	<code>npm run test:run</code>	L'ensemble des tests unitaires (dont sécurité) passe — vue d'ensemble	ST-AUTO / ST-SEC02
5	<code>npm run lint</code>	Qualité du code (réduit les motifs à risque)	ST-AUTO-01

Démo « commentée » (copier-coller)

```
# 1) Tests unitaires de sécurité (sanitization, session, masquage RH)
npm run security:test

# 2) Audit des dépendances de production (bloquant en CI)
npm run audit:prod

# 3) En-têtes de sécurité du déploiement en production (ST-SEC01)
npm run security:headers
# -> cibler une autre URL : npm run security:headers https://staging--dirmhublot.netlify.app

# 4) (optionnel) Scan de secrets local, si gitleaks est installé
# brew install gitleaks # macOS
gitleaks detect --no-git --config .gitleaks.toml --redact
```

À montrer sur GitHub (sans commande)

- Onglet **Actions** : workflows `CI` , `CodeQL (SAST)` , `Scans sécurité` en succès.
- Onglet **Security** → **Code scanning** : alertes CodeQL (SAST).
- Onglet **Security** → **Dependabot** : PR de mise à jour des dépendances.

Les scans **CodeQL**, **Gitleaks** et **Trivy** s'exécutent dans GitHub Actions (pas en local) ; pour la démo, on **montre l'onglet Actions/Security** plutôt que de les relancer.

6. Couverture par les scénarios de test

Scénario	Intention	Outil
ST-SEC01	En-têtes HTTP de sécurité présents en production	<code>curl</code> / <code>security:headers</code>
ST-SEC02	Les garde-fous applicatifs (sanitization, validation, session) fonctionnent	Vitest <code>security.test.ts</code>
ST-SEC03	Aucun secret commité dans le dépôt	Gitleaks
ST-SEC04	Aucune dépendance de production vulnérable non maîtrisée	<code>audit:prod</code> + Trivy

Scénario	Intention	Outil
ST-SEC05	Aucune faille de code détectée par l'analyse statique	CodeQL
ST-F01	Accès refusé sans identifiants valides	Playwright

7. Limites connues et évolutions

Constat	Évolution proposée
CSP absente sur Netlify (PROD sert HSTS + X-Frame-Options + nosniff, mais pas de <code>Content-Security-Policy</code>)	Aligner <code>netlify.toml</code> sur la CSP déjà présente dans <code>nginx.conf</code>
Authentification côté client (identifiants injectés au build via <code>VITE_*</code>)	Migrer vers une auth serveur (JWT / SSO)
Pas de DAST (test dynamique)	Ajouter un scan OWASP ZAP (baseline) sur l'URL de préproduction
CORS <code>*</code> sur la fonction Neon	Restreindre l'origine à l'URL du front
Trivy informatif	Passer en bloquant une fois la base de vulnérabilités triée

8. Correspondance OWASP (repère)

Catégorie OWASP	Couverture Hublot
A01 Broken Access Control	Auth applicative + tests login (Playwright)
A02 Cryptographic Failures	HTTPS/TLS (Netlify), HSTS
A03 Injection / XSS	<code>sanitizeInput</code> + tests Vitest + SAST CodeQL
A05 Security Misconfiguration	En-têtes HTTP + ST-SEC01 (<code>curl</code>)

Catégorie OWASP	Couverture Hublot
A06 Vulnerable Components	<code>audit:prod</code> + Dependabot + Trivy
A07 Identification & Auth Failures	Session + expiration (tests Vitest)
Secrets exposure	Gitleaks + <code>.gitignore</code> + variables d'environnement

9. Glossaire

Tous les termes et sigles employés dans ce document, expliqués simplement.

Stratégies et concepts

Terme	Définition
Test de sécurité	Test qui cherche activement les failles (vs test fonctionnel qui vérifie que ça marche).
Shift-left	« Décaler à gauche » : tester au plus tôt (dès le commit / la CI) plutôt qu'à la fin.
Défense en profondeur	Empiler plusieurs protections/outils complémentaires : si l'un laisse passer, un autre rattrape.
CI/CD	Continuous Integration / Continuous Deployment : chaîne automatisée qui teste (CI) puis déploie (CD) à chaque modification.
Pipeline	Suite d'étapes automatiques exécutées par la CI/CD (lint → tests → audit → build...).
Allowlist	Liste blanche : éléments explicitement autorisés/ignorés (ici, faux positifs assumés).
Faux positif	Alerte déclenchée à tort (ex. un identifiant de démo détecté comme « secret »).
Bloquant	Une étape qui fait échouer la livraison si elle ne passe pas.

Terme	Définition
-------	------------

SARIF Format standard de résultats d'analyse de sécurité, lu par l'onglet Security de GitHub.

Familles de tests

Sigle	Signifie	En clair
SCA	Software Composition Analysis	Analyse des dépendances (librairies tierces) pour repérer les failles connues.
SAST	Static Application Security Testing	Analyse du code source sans l'exécuter, à la recherche de motifs dangereux.
DAST	Dynamic Application Security Testing	Test de l'application en cours d'exécution (on l'attaque pour de vrai).
Secret scanning	—	Recherche de secrets (mots de passe, clés, tokens) commités par erreur.
Test unitaire	—	Test d'une petite fonction isolée (ici : sanitization, session).
Test fonctionnel / E2E	End-to-End	Test du parcours utilisateur complet dans un vrai navigateur.

Outils

Outil	Rôle
npm audit	Commande qui liste les vulnérabilités (CVE) des dépendances npm.
<code>audit:prod</code>	Script du projet : audit npm filtré (critical bloquant, high via allowlist).

Dependabot

Outil	Rôle
	Robot GitHub qui ouvre des PR pour mettre à jour les dépendances.
Trivy	Scanner de vulnérabilités (dépendances, fichiers, images).
CodeQL	Moteur d'analyse statique (SAST) de GitHub pour le code JS/TS.
Gitleaks	Outil de détection de secrets dans le code et l'historique Git.
Vitest	Framework de tests unitaires (utilisé pour <code>security.test.ts</code>).
Playwright	Outil de tests E2E pilotant un navigateur (login, session).
ESLint	Analyseur de qualité de code (lint).
curl	Outil en ligne de commande pour interroger une URL (ici : lire les en-têtes HTTP).
OWASP	Open Worldwide Application Security Project : référence en sécurité applicative (ex. le « Top 10 » des risques).
OWASP ZAP	Outil DAST gratuit de l'OWASP pour scanner une appli en fonctionnement.

Menaces et protections

Terme	Définition
Vulnérabilité	Faille exploitable par un attaquant.
CVE	Common Vulnerabilities and Exposures : identifiant public d'une vulnérabilité connue (ex. <code>CVE-2024-XXXX</code>).
XSS	Cross-Site Scripting : injection de code (JavaScript) dans une page pour pirater l'utilisateur.
Injection	Insertion d'entrées malveillantes (SQL, HTML, JS) exploitées par l'application.

Sanitization

Terme	Définition
	Nettoyage d'une entrée utilisateur pour neutraliser le code dangereux.
Clickjacking	Piéger l'utilisateur en superposant un site invisible (contré par <code>X-Frame-Options</code>).
MIME sniffing	Le navigateur « devine » le type d'un fichier et peut l'exécuter à tort (contré par <code>nosniff</code>).
CORS	Cross-Origin Resource Sharing : règles autorisant (ou non) un autre domaine à appeler une API.

En-têtes HTTP de sécurité

En-tête	Protège contre
HTTPS / TLS	L'interception des données (chiffrement de la connexion).
HSTS (<code>Strict-Transport-Security</code>)	Forcer HTTPS et empêcher la bascule en HTTP non sécurisé.
CSP (<code>Content-Security-Policy</code>)	Le XSS : restreint les sources de scripts/ styles autorisées.
<code>X-Frame-Options</code>	Le clickjacking (interdit l'affichage du site dans une iframe).
<code>X-Content-Type-Options: nosniff</code>	Le MIME sniffing.
<code>Referrer-Policy</code>	La fuite d'informations via l'en-tête <code>Referer</code> .
<code>Permissions-Policy</code>	L'usage non désiré de la caméra, du micro, de la géolocalisation.

Authentification

Terme	Définition
Session	

Terme	Définition
	État « connecté » conservé côté navigateur (ici <code>sessionStorage</code> , expire après 8 h).
JWT	JSON Web Token : jeton signé prouvant l'identité d'un utilisateur (auth serveur).
SSO	Single Sign-On : authentification unique partagée entre plusieurs applications.
Variable d'environnement	Valeur (secret, config) fournie hors code, jamais commitée (ex. <code>VITE_*</code> , secrets Netlify).

Synthèse

« Les tests de sécurité de Hublot suivent une stratégie shift-left et de défense en profondeur : à chaque intégration, la CI croise plusieurs familles d'outils — **SCA** (npm audit bloquant, Dependabot, Trivy), **SAST** (CodeQL), **scan de secrets** (Gitleaks), **tests unitaires de sécurité** (Vitest sur la sanitization et la session) et **tests fonctionnels d'accès** (Playwright). La configuration de déploiement est validée par un contrôle automatisé des en-têtes HTTP (ST-SEC01). Ce qui est certain bloque la livraison ; ce qui est à trier remonte dans l'onglet Sécurité du dépôt. »